

Interim Progress: Implementation of Classes

Overview

In this deliverable, you will begin implementing your project in C++. You are expected to implement two core OOP concepts: **Encapsulation** and **Association** (both aggregation and composition). Base your implementation on the UML class diagram and design decisions from Deliverable 2.

This deliverable consists of **three (3) submission items**: (1) the **project source code**, (2) a **presentation video**, and (3) the **Teamwork Assessment 3 (TW3)/ Peer and Self-Assessment (PSA)**.

Requirements

Review your UML class diagram from Deliverable 2 and implement the following in C++:

1. **Implement all classes with encapsulation.** Each class must include the required class members (see Task 1 below for details).
2. **Implement the association relationships.** Identify which classes in your design are connected by an association relationship and implement **both aggregation and composition** in your code.
3. **Test each class individually.** You do not need to integrate all classes into a complete working system at this stage. Test each class separately in its own main file.

Note on Inheritance and Polymorphism:

Inheritance and polymorphism are **not required** in this deliverable.

For inheritance: implement the **parent (base) class only**. You do not need to implement derived classes yet.

For polymorphism: if your UML design includes virtual methods, treat them as **regular methods** (without the `virtual` keyword) for now.

Task 1: Project Source Code

Submit your source code as a **compressed ZIP file**. Your code must compile and run without errors.

What to Include

The ZIP file must contain the following files:

1. **Class header files (*.hpp):** one header file per class, containing the class declaration.
2. **Class implementation files (*.cpp):** one implementation file per class, containing the function definitions.
3. **Test or main files:** one or more main files used to test each class individually (e.g., `main1.cpp`, `main2.cpp`, etc.).
4. **Input files (if any):** any data files (.txt or similar) that your program reads from.

★ File naming convention:

Please name your files clearly using the format below, so it is easy to identify your class:

Header file: `ClassName.hpp` (e.g., `Student.hpp`, `ActivityLog.hpp`)

Implementation file: `ClassName.cpp` (e.g., `Student.cpp`, `ActivityLog.cpp`)

Test/main file: `testClassName.cpp` (e.g., `testStudent.cpp`, `testActivityLog.cpp`)

★ **ZIP file naming convention:** `GroupName_Section_D3.zip` (e.g., `GroupAlpha_Sect04_D3.zip`)

Requirements for Each Class

Each class in your system must implement the following:

1. **Attributes:** all data members declared as `private`.
2. **Constructor and Destructor:** a constructor that initialises all attributes, and a destructor.
3. **Accessor and Mutator functions:** getter and setter functions for **all attributes**. You may add additional utility methods as needed.
4. **Other helper or utility methods:** any additional methods that serve the purpose of the class, such as `displayInfo()`, `calculateTotal()`, or similar.

★ What makes a good implementation:

- Use initialiser lists in constructors where appropriate.
- All public methods that do not modify data must be declared `const`.
- Use the scope resolution operator (`::`) for all function definitions outside the class.
- Avoid hardcoded values: use named constants or parameters instead.
- Include a brief comment above each function explaining what it does.

Implementing Association Relationships

Based on your class diagram from Deliverable 2, identify which classes are in an association relationship and implement **both**:

1. **Composition:** one class owns another. The contained object is created and destroyed together with the owner. Implemented using **an object member** (not a pointer) inside the owner class.
2. **Aggregation:** one class uses another, but the two exist independently. Implemented using **a pointer or a reference** to the other class.

★ Implementation tip:

In your test/main file, demonstrate the association relationship clearly. For composition, show that the contained object is created along with the owner. For aggregation, show that the two objects exist separately and the relationship is set through a pointer or a method call.

Testing Your Code

At this stage, you are **not required** to build a complete working system. Instead, test each class individually in a separate main file. For each class, your test file should:

4. **Create an object** (or an array of objects) of the class.
5. **Call the key methods** of the class to demonstrate that it works correctly. Focus on the important methods, those that serve the main purpose of the class.

6. Display the output to show the result of each method call.

Example:

If your project has a class named `Student`, your test file would:

- Create a `Student` object or an array of `Student` objects.
- Set the attributes using the constructor or mutator functions.
- Call methods such as `displayInfo()` or `calculateGPA()` to show the class works.
- Print the output to the console.

Repeat the same steps in a separate test file for each additional class in your system.

Task 2: Presentation Video

This deliverable is assessed **mainly through the video**. Prepare your video carefully. The video is a **per-member presentation**, each member presents their **own work only**.

What to Present

For your own module/classes, present the following in your video:

1. **Show your class implementation.** Walk through your header and implementation files. Explain the attributes, constructor, destructor, and key methods.
2. **Demonstrate the association relationship.** If your class is involved in an aggregation or composition relationship, show the code and explain how the relationship is implemented.
3. **Run the test/demo.** Show your test file running. Walk through the output and explain what each method call demonstrates.

★ What makes a good video presentation:

- Show your VS Code (or IDE) with a font size large enough to read clearly on screen.
- Explain your code, do not just read it out. Say why you made certain design decisions.
- For each method you call in the demo, briefly explain what it does and what the output means.
- Show both the code and the console output at the same time if possible.
- Speak clearly and ensure the audio is audible throughout.

Where to Host the Video

1. The video must be uploaded to YouTube.
2. Share only the video URL when making your submission on e-Learning.
3. Ensure the video remains publicly accessible until you receive your final grade for this course.
4. Do not submit the video file directly on e-Learning.

Timestamps

1. Include timestamps in the YouTube video description box.
2. Each timestamp must state the time, the topic or segment, and the name of the presenting member.

Example:

```
00:00:01 Class User: implementation and demo (Ahmad Rizal)
03:00:00 Class AdminUser: implementation and demo (Nur Aisyah)
06:30:00 Association relationship (composition/aggregation) demo (Lim Wei Xian)
```

Single or Multiple Videos

1. It is recommended to present all segments in a single video.
2. Separate videos are acceptable. If so, provide all video URLs in your submission and include links between videos in the YouTube description box.

Video Duration

1. Keep the video as concise as possible.
2. **Do not exceed 15 minutes** in total.

Who Should Present

1. All group members must appear and present in the video.
2. Each member must show their face on camera.
3. Each member presents only their own work; not the work of other members.
4. **Any member who does not present will receive a ZERO mark** for this deliverable.

★ Divide the classes among group members clearly. Each member is responsible for demonstrating the classes they implemented. If a class involves an association with a classmate's class, coordinate to show the relationship in one member's segment and cross-reference it.

Video Recording Tools

1. Online meeting software such as Google Meet, Webex, or Zoom (using the built-in recording feature).
2. OBS Studio (Open Broadcaster Software) suitable for screen recording or screencasting.

Teamwork Assessment 3 (TW3)

Each member must individually complete and submit **Teamwork Assessment 3 (TW3)**, which is a Peer and Self-Assessment (PSA). This is an **individual submission**; it is not submitted as a group.

Important Notes for PSA:

- Download a copy of the Excel file. Do not edit it directly online (on my Google Drive).
- Edit on your own local copy.
- Submit your copy to e-Learning.
- Use a separate copy for each PSA submission.
- Do not leave any empty fields. You must also rate yourself.
- Video guide on how to fill in the form:
- https://drive.google.com/file/d/1E4xWmhkQo5ZMiGe3nR015i8Uub0wFeEN/view?usp=drive_link
- You may watch the video at 1.5× speed to save time (use the playback speed option in the video player settings).

Assessment

This deliverable carries **6%** weightage for the final grade of this course. The breakdown is as follows:

Criteria	Weightage
1. Implementation of Encapsulation:	1.5 %

Criteria	Weightage
- Classes declared with correct data members (private attributes). - Constructor and destructor implemented for each class. - Accessor (getter) and mutator (setter) functions implemented for all attributes. - Additional helper or utility methods included where appropriate. <i>Assessed individually per member based on their own classes shown in the video.</i>	
2. Implementation of Association: - Both aggregation and composition are implemented correctly. - Code clearly demonstrates the difference between the two relationship types. <i>Justification of relationship type (why aggregation vs composition) is part of the video presentation.</i>	1.0 %
3. Code Testing and Class Demonstration: Each member demonstrates their own classes individually in the video. For each class, the demonstration must show: - Creating objects (or an array of objects) of the class. - Calling key methods and showing the output. - Demonstrating that the class serves its intended purpose. <i>Marks are awarded individually based on each member's demonstration in the video.</i>	1.5 %
4. Code Organisation: - Code is properly structured using separate .hpp and .cpp files. - File names clearly reflect the class they contain. - Code is readable: consistent indentation, proper use of <code>const</code> and <code>::</code>, and brief comments. <i>Assessed from both the submitted source code and the code visible in the video.</i>	1.0 %
Teamwork Assessment 3 (TW3) - Peer and Self-Assessment: Individual submission. Evaluated by peers, yourself, and the lecturer. The lecturer's assessment is based on your involvement and contribution visible in the video. Failure to submit will result in a ZERO mark.	1.0 %
Total	6.0 %

Submission

- Deadline:** Week 12, Friday, 5 June 2026, 23:59 MYT (UTC+8).
- Late submission penalty:** 10% deduction for every 1 day late.

Group Submission (Video)

- Submit the YouTube video URL to e-Learning.
- Only one (1) member per group needs to make this submission.

Group Submission (Source Code)

- Submit the compressed source code and input files as a **ZIP file** to e-Learning.
- Name the ZIP file as follows: `GroupName_Section_D3.zip`

(e.g., GroupAlpha_Sect04_D3.zip).

3. Only one (1) member per group needs to make this submission.

Individual Submission (TW3/ PSA)

1. Each member must individually submit TW3 (Peer and Self-Assessment) to e-Learning.
2. **Failure to submit will result in a ZERO mark for TW3.**

FAQs

1. Does the program need to be fully working?

The program must compile and run without errors, and all implemented classes must work correctly and serve their intended purpose. A complete end-to-end working system is **not required** at this stage.

2. How do we test the code if the system is not complete yet?

Test each class individually in a separate main file. Create objects of the class, call its key methods, and display the output. You do not need to integrate all classes together yet.

Example: if your project has a class named `ActivityLog`, create a test file `testActivityLog.cpp`. In `main()`, create an `ActivityLog` object, call `addRecord()`, `displayAll()`, and `getRecordCount()`, and print the results.

3. Do we need to implement both aggregation and composition?

Yes. Both must be implemented. Refer to your UML class diagram from Deliverable 2 to identify which of your classes have these relationships.

4. How do we handle inheritance in this deliverable?

Implement the **parent (base) class only**. You do not need to implement derived classes at this stage.

5. How do we handle polymorphism in this deliverable?

If your UML design has virtual methods, implement them as **regular methods** (without the `virtual` keyword) for now. Polymorphism will be implemented in Deliverable 4.

6. Can we include inheritance if we choose to?

Yes, but it is not required. If you include it, do so only for code reusability purposes. **No additional marks** will be awarded for inheritance in this deliverable.

7. How do we divide the work among group members?

It is recommended to **split the work by class**. Each member takes responsibility for implementing and testing their own classes. This is reflected in the individual assessment; each member presents their own work in the video.

8. Will marks be given individually or as a group?

Both. Code organisation (criterion 4) and the source code submission are assessed as a group. Encapsulation (criterion 1), code testing (criterion 3), and TW3/PSA are assessed **individually** based on each member's video presentation and TW3 submission.

9. Can we use code samples provided by the lecturer on e-Learning?

You may refer to the samples for understanding. However, you are not permitted to reuse them as-is in your submission. Your code must be your own work.